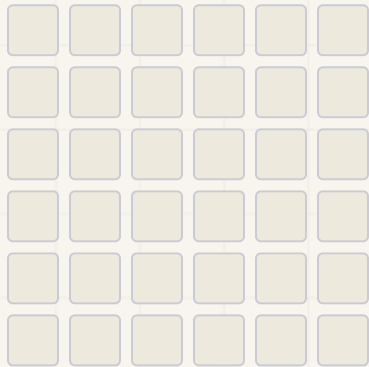


Aula 2

Extensões e a camada transformer completa

De self-attention puro a um bloco pronto para empilhar — positional encoding, scaled dot-product, multi-head attention e a camada transformer completa.



UDL CAP. 12 · PRINCE SEÇÕES 12.3 – 12.4 EQ. 12.9 – 12.13

Das extensões à camada completa

A Aula 1 chegou até $\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q}]$. Hoje adicionamos três extensões quase sempre usadas na prática, e fechamos a camada transformer inteira.

12.3.1	Positional encoding	absoluto & relativo	12.4	Camada transformer completa	eq. 12.13
12.3.2	Scaled dot-product attention	eq. 12.9	→	Recapitulação em cadeia	eq. 12.9 – 12.13
12.3.3	Multi-head attention	eq. 12.10 – 12.12	→	Fechamento e próximos passos	o que fica para depois

O que falta: noção de ordem

Self-attention "puro" (eq. 12.8) é equivariante a permutações — não leva em conta a ordem dos inputs $\mathbf{x}_n$. Mas em linguagem, ordem carrega significado.

significado A

A mulher comeu o guaxinim.

significado B

O guaxinim comeu a mulher.

Problem 12.3 – propriedade formal

$$\text{Sa}[\mathbf{X}\mathbf{P}] = \text{Sa}[\mathbf{X}] \mathbf{P}$$

para qualquer matriz de permutação $\mathbf{P}$ — trocar a ordem das colunas de $\mathbf{X}$ apenas reordena as saídas correspondentes, sem alterar seus valores. Isso mesmo é o problema: as duas frases acima teriam os *mesmos vetores de saída*, só reordenados.

Somar uma marca de posição

A solução mais direta: somar uma matriz Π ao input, onde cada coluna é única e codifica a posição absoluta (Fig. 12.5 do livro).

$$\mathbf{X} \leftarrow \mathbf{X} + \Pi$$

Π tem uma coluna distinta para cada posição — o padrão pode ser senoidal (fixo, escolhido à mão) ou aprendido como qualquer outro parâmetro.

ONDE SOMAR

Pode ser somada apenas nos embeddings de entrada, ou em cada camada do transformer.

UM DETALHE SUTIL

Às vezes é somada apenas no cálculo de Q e K — não em V .

O que importa é a distância, não o lugar

A posição absoluta de uma palavra é menos importante que a posição relativa entre duas palavras — um trecho de texto pode começar em qualquer ponto de um documento.

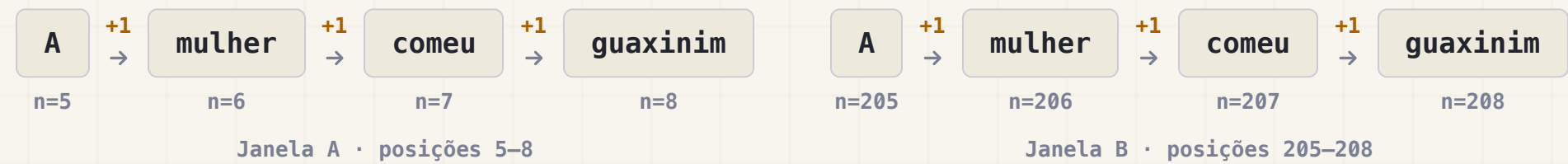
$$\mathbf{A}_{m,n} \leftarrow \mathbf{A}_{m,n} + \pi_{m-n}$$

Um parâmetro $\pi_{a,b}$ aprendido para cada offset entre a posição da key (a) e da query (b) — modifica a matriz de atenção somando, multiplicando, ou de outra forma.

Captura algo como "*a palavra três posições atrás*", em vez de "*a palavra na posição 7*" — mais robusto quando o mesmo texto pode ser fatiado em janelas diferentes.

A mesma frase, em janelas diferentes

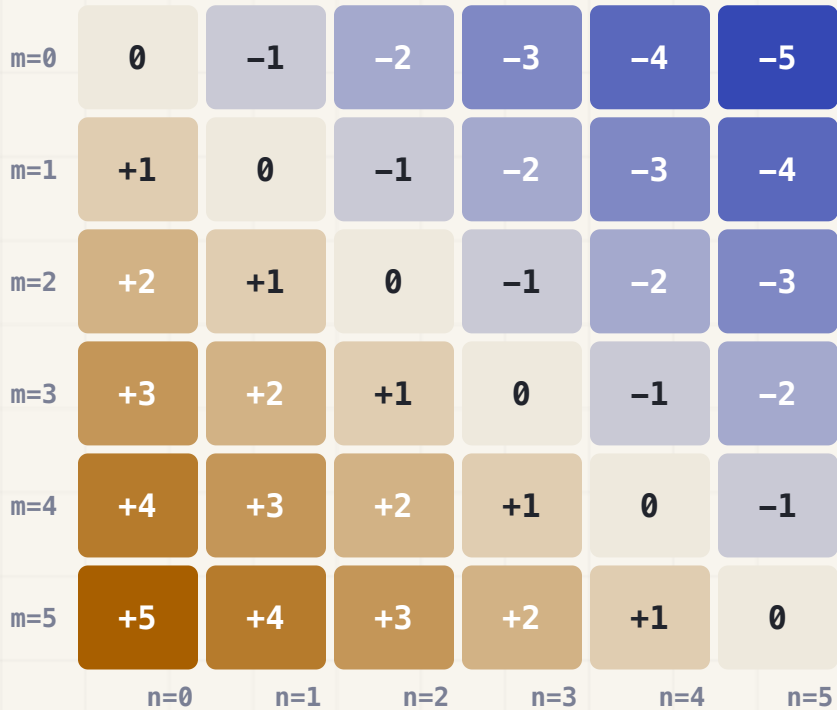
Volte à frase do início da aula: "*A mulher comeu o guaxinim.*" O que carrega o significado é a ordem relativa entre as palavras — e essa relação não muda se a frase aparece perto do início ou no meio de um documento longo.



Um encoding absoluto daria à "mulher" da janela A e à "mulher" da janela B duas marcas totalmente diferentes — o modelo teria que reaprender o mesmo padrão em cada faixa de posições. Um encoding **relativo** só enxerga a distância +1 nos dois casos — o mesmo π_1 aprendido serve para o documento inteiro.

A matriz de atenção, colorida por deslocamento

Cada célula $A_{m,n}$ recebe π_{m-n} — e como o valor só depende da diferença $m - n$, toda a diagonal compartilha o mesmo número.



m = posição da query (linha) · n = posição da key (coluna) · mesma cor = mesma diagonal = mesmo π_{m-n}

Por que os produtos escalares precisam de freio

Produtos escalares podem ter magnitude grande, empurrando o softmax para uma região onde o maior valor domina completamente (Problem 12.4).

exemplo – cinco pré-ativações

$$\mathbf{z} = [-3, 1, 100, 5, -1]$$
$$\text{softmax}(\mathbf{z}) \approx [0, 0, 1, 0, 0]$$

$z_3 = 100$ domina inteiramente. Pequenas mudanças em qualquer z_i (inclusive z_3) têm efeito quase nulo na saída — os 25 gradientes $\partial y_i / \partial z_j$ ficam próximos de zero. O modelo praticamente para de aprender ali.

H conjuntos de Q, K, V em paralelo

Em vez de um único self-attention, calculamos H deles simultaneamente — cada "head" com seus próprios parâmetros.

EQ. 12.10 — para cada head h

$$\mathbf{V}_h = \beta_{vh} \mathbf{1}^T + \Omega_{vh} \mathbf{X}$$

$$\mathbf{Q}_h = \beta_{qh} \mathbf{1}^T + \Omega_{qh} \mathbf{X}$$

$$\mathbf{K}_h = \beta_{kh} \mathbf{1}^T + \Omega_{kh} \mathbf{X}$$

Mesma matriz $\mathbf{X}$ de entrada para todas as heads — só os parâmetros $\{\beta_{\bullet h}, \Omega_{\bullet h}\}$ diferem entre elas.

Cada head roda seu próprio scaled attention

A mesma eq. 12.9 de antes, aplicada de forma independente dentro de cada head.

EQ. 12.11

$$\mathbf{S}a_h[\mathbf{X}] = \mathbf{V}_h \cdot \text{Softmax} \left[\frac{\mathbf{K}_h^T \mathbf{Q}_h}{\sqrt{D_q}} \right]$$

Parâmetros $\{\beta_{vh}, \Omega_{vh}\}, \{\beta_{qh}, \Omega_{qh}\}, \{\beta_{kh}, \Omega_{kh}\}$ — distintos para cada head h .

Concatenar e recombina

As saídas das H heads são empilhadas verticalmente, e uma transformação linear Ω_c as recombina (Fig. 12.6).

EQ. 12.12

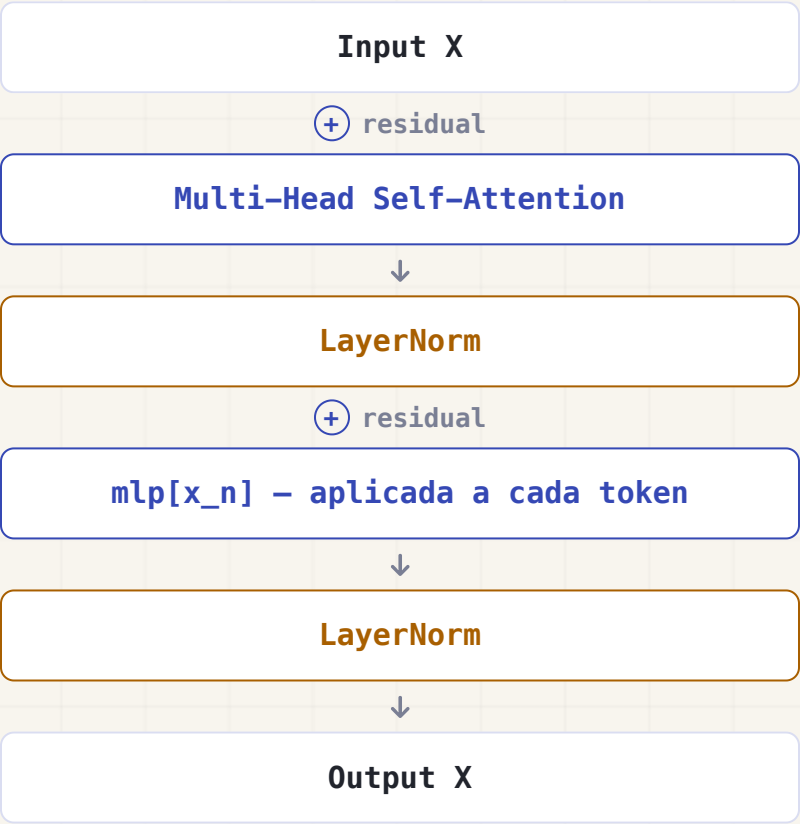
$$\text{MhSa}[\mathbf{X}] = \Omega_c [\text{Sa}_1[\mathbf{X}]^T, \text{Sa}_2[\mathbf{X}]^T, \dots, \text{Sa}_H[\mathbf{X}]^T]^T$$

Se a entrada tem dimensão D e há H heads, tipicamente values/queries/keys de cada head têm tamanho D/H — o que torna a implementação eficiente (Problem 12.5).

Múltiplas heads parecem necessárias para o self-attention funcionar bem na prática — especula-se que tornam a rede mais robusta a más inicializações.

A camada completa, peça por peça

Self-attention é só uma parte. A camada transformer soma multi-head attention e uma rede totalmente conectada (aplicada a cada token separadamente), cada uma como bloco residual, com LayerNorm depois de cada uma (Fig. 12.7).



A camada completa, em quatro linhas

LayerNorm normaliza cada embedding usando estatísticas calculadas ao longo das suas próprias D dimensões — diferente de BatchNorm, que usa estatísticas do batch.

EQ. 12.13

$$\begin{aligned} \mathbf{X} &\leftarrow \mathbf{X} + \text{MhSa}[\mathbf{X}] \\ \mathbf{X} &\leftarrow \text{LayerNorm}[\mathbf{X}] \\ \mathbf{x}_n &\leftarrow \mathbf{x}_n + \text{mlp}[\mathbf{x}_n] \quad \forall n \in \{1, \dots, N\} \\ \mathbf{X} &\leftarrow \text{LayerNorm}[\mathbf{X}] \end{aligned}$$

Os vetores-coluna $\mathbf{x}_n$ são tomados separadamente da matriz completa $\mathbf{X}$ para a etapa do mlp — a mesma rede é aplicada a cada token, independentemente. Numa rede real, os dados passam por uma série dessas camadas.

A cadeia completa, eq. 12.9 $\rightarrow$ 12.13

De self-attention puro até um bloco pronto para empilhar N vezes.

12.9	$\text{Sa}[\mathbf{X}] = \mathbf{V} \cdot \text{Softmax}[\mathbf{K}^T \mathbf{Q} / \sqrt{D_q}]$	scaled dot-product
12.10	$\mathbf{V}_h, \mathbf{Q}_h, \mathbf{K}_h$ — parâmetros por head	setup multi-head
12.11	$\text{Sa}_h[\mathbf{X}] = \mathbf{V}_h \cdot \text{Softmax}[\mathbf{K}_h^T \mathbf{Q}_h / \sqrt{D_q}]$	attention por head
12.12	$\text{MhSa}[\mathbf{X}] = \Omega_c[\text{Sa}_1[\mathbf{X}]^T, \dots, \text{Sa}_H[\mathbf{X}]^T]^T$	concatenar heads
12.13	$\mathbf{X} \leftarrow \mathbf{X} + \text{MhSa}[\mathbf{X}]; \text{LayerNorm}; +\text{mlp}; \text{LayerNorm}$	camada completa

O que fica de pé até aqui

Temos agora um bloco transformer completo: positional encoding devolve a noção de ordem, scaled dot-product evita a saturação do softmax, multi-head permite capturar diferentes tipos de relação em paralelo, e a camada com resíduos + LayerNorm é estável o suficiente para empilhar muitas vezes.

Ficou de fora, de propósito: como esse bloco se encaixa num pipeline de NLP completo — tokenização, embeddings e os três tipos de modelo (encoder, decoder, encoder-decoder) que o resto do capítulo constrói em cima disso.